

Part2,3

이나연

Part2

(a) - (i) Momentum의 효과

- 이전 step에서의 파라미터 학습 방향을 어느정도 반영해 새 학습 방향을 정합니다. 업데이트의 분산이 줄어들고, 지속적으로 일관된 방향으로의 momentum이 붙어 더 빠르고 안정적으로 이동할 수 있습니다.

(a) - (ii) Adaptive Learning Rate 효과

- 그래디언드가 작거나 드물게 업데이트한 파라미터는 분모가 작아져 더 큰 업데이트가 가능하고, 기울기가 크고 빈번하게 업데이트된 파라미터는 분모가 커져 업데이트의 폭이 줄어듭니다. 희소항 특징에 대응되는 파라미터도 충분히 학습될 수 있도록 하며, 전반적인 파라미터들의 학습 균형을 맞춰줍니다.

(b) - (i) Inverted Dropout의 γ 값 유도

- Dropout은 hidden layer의 일부 원소를 p_{drop} 의 확률에 따라 0으로 만들고, 살아남은 원소에 대해서는 γ 상수를 곱합니다. 살아남은 항들은 $(1-p_{\text{drop}})$ 확률을 가지며, 아래와 같습니다.

$$\mathbb{E}[\mathbf{h}_{\text{drop}}]_i = \gamma \mathbb{E}[d_i] h_i = \gamma (1 - p_{\text{drop}}) h_i$$

따라서 $\gamma = 1/(1-p_{\text{drop}})$ 입니다.

(b) - (ii) Dropout을 Train에는 적용하고, Evaluation에는 적용하지 않아야 하는 이유

- 훈련 시에는 뉴런을 무작위로 제거함으로써 특정 뉴런 간에 과도하게 의존하는 현상을 방지합니다.
- 또한 수많은 네트워크를 학습시키는 일종의 앙상블 효과를 통해 과적합을 방지합니다.
- 대신 평가 시에는 모든 뉴런과 파라미터를 온전히 활용하여, 매번 결과가 달라지지 않고 결정론적으로 안정된 최선의 예측 성능을 내야 하기 때문에 dropout을 적용하지 않습니다.

Part3

(a) parsing 표 채우기

Stack(처리중)	Buffer(대기열)	New dependency	Transition
[ROOT]	[I, presented, my, findings, at, the, NLP, conference]	[]	
[ROOT, I]	[presented, my, findings, at, the, NLP, conference]	[]	SHIFT
[ROOT, I, presented]	[my, findings, at, the, NLP, conference]	[]	SHIFT
[ROOT, presented]	[findings, at, the, NLP, conference]	(presented → I)	LEFT-ARC
[ROOT, presented, my]	[at, the, NLP, conference]	[]	SHIFT
[ROOT, presented, my, findings]	[at, the, NLP, conference]	[]	SHIFT
[ROOT, presented, findings]	[at, the, NLP, conference]	(findings → my)	LEFT-ARC
[ROOT, presented, presented]	[at, the, NLP, conference]	(presented → findings)	RIGHT-ARC
[ROOT, presented, at]	[the, NLP, conference]	[]	SHIFT
[ROOT, presented, at, the]	[NLP, conference]	[]	SHIFT
[ROOT, presented, at, the, NLP]	[conference]	[]	SHIFT
[ROOT, presented, at, the, NLP, conference]	[]	[]	SHIFT

[ROOT, presented, at, the, conference]	[]	(conference → NLP)	LEFT-ARC
ROOT, presented, at, conference]	[]	(conference → the)	LEFT-ARC
[ROOT, presented, conference]	[]	(conference → at)	LEFT-ARC
[ROOT, presented]	[]	(presented → conference)	RIGHT-ARC
[ROOT]	[]	(ROOT → presented)	LEFT-ARC

(b) n개의 단어 가진 문장 파싱 몇 번 필요한지 n에 대해 나타내고 이유 설명하기

- 답 : 2n steps
- 설명 : 버퍼에 있는 n개의 단어가 스택으로 올라오기 위해서는 n번의 shift가 필요하고, 파싱이 완료된 후에는 스택에 있는 n개의 단어가 LEFT-ARC 또는 RIGHT-ARC를 통해 각각 한 번씩은 스택에서 제거되어야 하기 때문에, n번의 ARC 연산이 필요하므로 총 2n 단계가 필요합니다.

(c)

```
SHIFT test passed!
LEFT-ARC test passed!
RIGHT-ARC test passed!
parse test passed!
```

(d)

```
minibatch_parse test passed!
```

(e) - i

$$\frac{\partial h_i}{\partial x_j} = W_{ji} \cdot \mathbf{1}\{z_i > 0\}$$

